Module Boundary Definition — pqc_crypto_module v0.1.0

> **Disclaimer**: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

---

1. Boundary Statement

The cryptographic boundary of `pqc_crypto_module` is defined as all Rust source files within `crates/pqc_crypto_module/src/`. Code outside this directory is outside the cryptographic boundary.

The Rust crate system provides a natural isolation boundary: external code can only access items explicitly marked `pub` in the crate's module tree. Internal functions marked `pub(crate)` are invisible to external code.

2. Files Inside the Boundary

File	Lines	Responsibility	Contains Crypto
`src/lib.rs`	~34	Crate root, module re-exports	No
`src/api.rs`	~82	Public API entry point, all approved operations	Delegates
`src/mldsa.rs`	~95	ML-DSA-65 key generation, signing, verification	Yes
`src/mlkem.rs`	~130	ML-KEM-768 key encapsulation (FIPS 203 via `pqcrypto-mlkem`)	Yes
`src/hashing.rs`	~41	SHA3-256 hashing	Yes
`src/rng.rs`	~61		Yes

File	Lines	Responsibility	Contains Crypto
		CSPRNG wrapper, continuous RNG test	
`src/self_tests.rs`	~100	Known Answer Tests	Yes (test vectors)
`src/approved_mode.rs`	~68	State machine, approved-mode guards	No (control logic)
`src/types.rs`	~123	Typed wrappers with ZeroizeOnDrop	No (data types)
`src/errors.rs`	~26	CryptoError enum	No (error types)
`src/legacy.rs`	~120	Non-approved algorithms (gated)	Yes (non-approved)

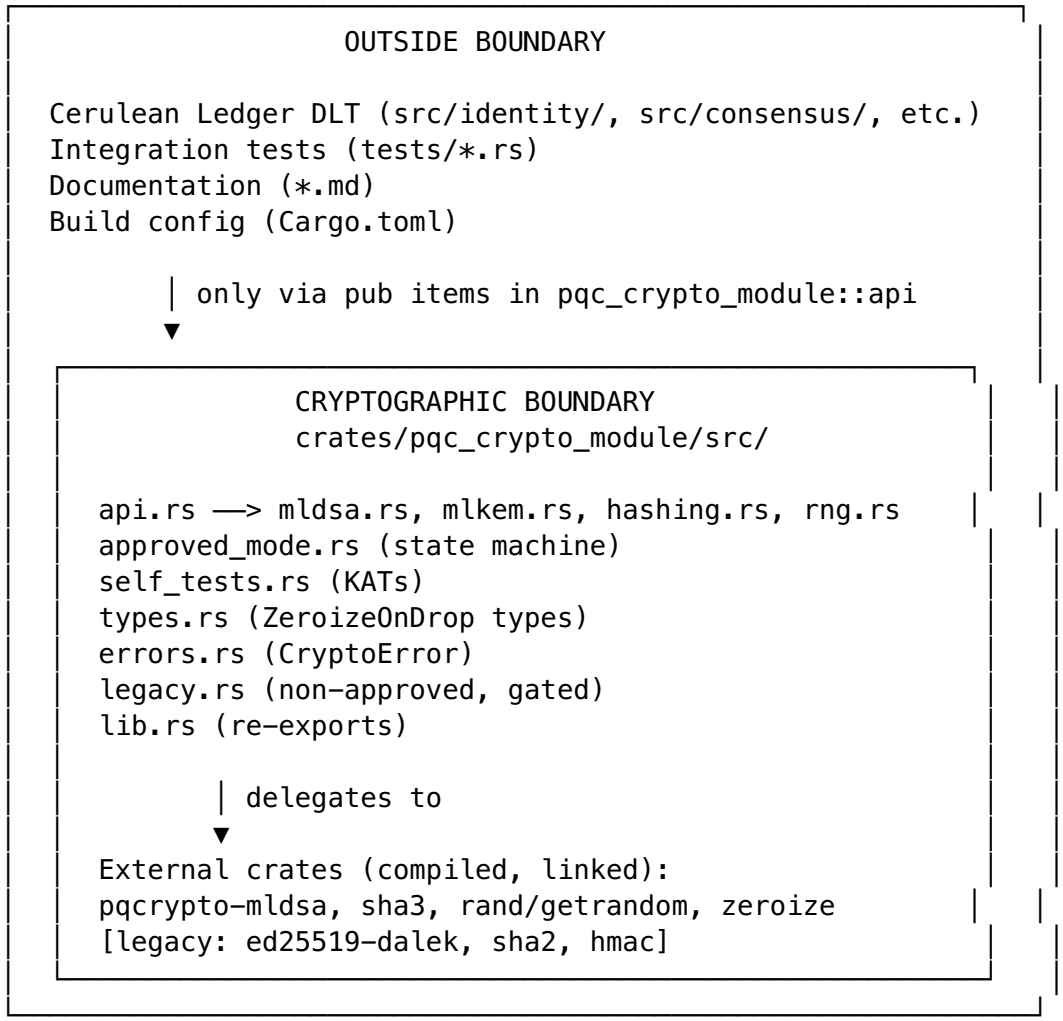
Total: 11 source files.

3. Files Outside the Boundary

Path	Role	Boundary Status
`Cargo.toml`	Build configuration and dependency declarations	Outside
`tests/api_boundary.rs`	Integration test: API behavior before/after init	Outside
`tests/approved_vs_legacy.rs`	Integration test: approved vs. legacy separation	Outside
`tests/no_fallback.rs`	Integration test: no classical algorithm fallback	Outside
`tests/self_tests.rs`	Integration test: self-test execution	Outside
`tests/key_zeroization.rs`	Integration test: key zeroization behavior	Outside
`SECURITY_POLICY.md`	Security policy document	Outside
`SECURITY_POLICY_DRAFT.md`	Draft security policy	Outside
`DESIGN_DOCUMENT.md`	Architecture document	Outside
`FINITE_STATE_MODEL.md`	State machine document	Outside
`KEY_MANAGEMENT.md`		Outside

Path	Role	Boundary Status
	Key management document	
SELF_TEST_DOCUMENTATION.md	Self-test document	Outside
NON_APPROVED_USAGE.md	Non-approved algorithm document	Outside
OPERATIONAL_GUIDANCE.md	Operational guidance	Outside
README.md	Crate readme	Outside
build/*.md	Build documentation	Outside

4. Boundary Diagram



5. Enforcement Mechanisms

5.1 Rust Crate System

The Rust compiler enforces that external code can only access `pub` items. Internal functions (e.g., `generate_keypair_raw()`, `sha3_256_raw()`) are `pub(crate)` and are invisible to code outside the crate.

5.2 API Entry Point Constraint

All approved cryptographic operations are accessed through `pqc_crypto_module::api`. This is enforced by convention and verified by integration tests.

5.3 Integration Tests

Test file	What it enforces
`tests/api_boundary.rs`	All operations fail before init (state gate works)
`tests/no_fallback.rs`	No Ed25519 or SHA-256 available through the approved API
`tests/approved_vs_legacy.rs`	Legacy blocked in Approved mode; no implicit fallback
`tests/key_zeroization.rs`	Key material is zeroized on drop

5.4 Compile-Time Exclusion

The `approved-only` feature flag causes `legacy.rs` to emit `compile_error!`, making it impossible to compile any code that references non-approved algorithms.

5.5 Boundary Verification Test

The boundary can be mechanically verified by listing the source files:

```
# All files inside the boundary
ls crates/pqc_crypto_module/src/*.rs

# Expected output:
# api.rs  approved_mode.rs  errors.rs  hashing.rs  legacy.rs
# lib.rs  mldsa.rs  mlkem.rs  rng.rs  self_tests.rs  types.rs
```

Any new `.rs` file added to `src/` is automatically inside the boundary and must be reviewed for FIPS compliance.

6. External Dependencies Within the Boundary

The following external crates are linked into the module and execute within the boundary:

Crate	Approved	Notes
`pqcrypto-mldsa`	Yes	Wraps PQClean C reference implementation
`pqcrypto-traits`	Yes	Trait definitions only
`sha3`	Yes	Pure Rust SHA3-256
`rand` + `rand_core`	Yes	OS-backed CSPRNG wrapper
`zeroize`	Yes	Memory zeroization
`thiserror`	N/A	Compile-time macro only
`hex`	N/A	Utility (hex encoding)
`ed25519-dalek`	No	Legacy only, gated
`sha2`	No	Legacy only, gated
`hmac`	No	Legacy only, gated

Non-approved dependencies (`ed25519-dalek`, `sha2`, `hmac`) are excluded from the boundary when compiled with `--features approved-only`.